Time Series Forecasting of Foreign Exchange Rates
A Comparative Analysis of Classical and Deep Learning Approaches

---

Student: [Your Name]
Course: Time Series Analysis
Date: October 2026
Theme: Finance - Foreign Exchange Rates
Assets: EUR/USD and GBP/USD
Data Period: January 1, 2015 to Present (≥5 years)

Table of Contents

1. [Introduction](#1-introduction)
2. [Data Description and Preprocessing](#2-data-description-and-preprocessing)
3. [Exploratory Data Analysis](#3-exploratory-data-analysis)
4. [Classical Time Series Models](#4-classical-time-series-models)
5. [Advanced Deep Learning Models](#5-advanced-deep-learning-models)
6. [Model Comparison and Evaluation](#6-model-comparison-and-evaluation)
7. [Results and Discussion](#7-results-and-discussion)
8. [LLM Contribution Assessment](#8-llm-contribution-assessment)
9. [Conclusion](#9-conclusion)
10. [References](#10-references)
11. [Appendices](#appendices)

---

1. Introduction

1.1 Problem Statement

Foreign exchange (FX) rate forecasting is a fundamental challenge in financial markets with significant implications for international trade, risk management, investment decisions, and monetary policy. The EUR/USD and GBP/USD currency pairs represent two of the most liquid and actively traded FX pairs globally, making them ideal candidates for time series analysis and forecasting research.

This project addresses the question: **Which time series modeling approach—classical statistical methods or modern deep learning techniques—provides the most accurate short-term forecasts for EUR/USD exchange rates?**

1.2 Research Objectives

The primary objectives of this study are:

1. **Implement and compare** classical time series models (ARIMA, VAR, VECM, GARCH) with advanced deep learning models (LSTM, GRU, Hybrid ARIMA+LSTM)
2. **Evaluate forecast accuracy** using multiple performance metrics (MSE, RMSE, MAE, MAPE)
3. **Assess statistical significance** of differences between model forecasts using the Diebold-Mariano test
4. **Validate results** across both R and Python implementations to ensure reproducibility
5. **Document the role** of Large Language Models (LLMs) in accelerating the development process

1.3 Selected Series and Modeling Questions

Primary Series: EUR/USD log returns
Secondary Series: GBP/USD log returns (used as exogenous variable in multivariate models)
Data Frequency: Daily
Sample Period: January 29, 2015 to October 17, 2026 (3,050 observations)
Forecast Horizon: 7 days ahead

Modeling Questions:
- Does EUR/USD exhibit volatility clustering that requires GARCH modeling?
- Are EUR/USD and GBP/USD cointegrated, suggesting a long-term equilibrium relationship?
- Can multivariate VAR models capture cross-series dependencies better than univariate ARIMA?
- Do deep learning models (LSTM, GRU) capture non-linear patterns that classical models miss?
- Does a hybrid approach combining ARIMA and LSTM improve forecast accuracy?

1.4 Methodological Overview

The project follows a structured approach:

1. **Data Collection:** Download daily OHLCV data from Yahoo Finance for EUR/USD and GBP/USD
2. **Preprocessing:** Calculate log returns, moving averages, and realized volatility
3. **Exploratory Analysis:** Stationarity tests, ACF/PACF analysis, STL decomposition, outlier detection
4. **Model Training:** Fit classical models in both R and Python, deep learning models in Python

```
5. **Forecasting:** Generate 7-day ahead forecasts using each model
6. **Evaluation:** Compare forecasts using error metrics and statistical tests
7. **Documentation:** Record all steps, including LLM-assisted development

---

## 2. Data Description and Preprocessing

### 2.1 Data Source and Collection

Data was obtained from Yahoo Finance using the `yfinance` Python package and
`quantmod` R package. The dataset includes:

- **EUR/USD:** 3,050 daily observations from January 29, 2015 to October 17, 2026
- **GBP/USD:** 3,050 daily observations from the same period
- **Variables:** Open, High, Low, Close, Volume for each currency pair

### 2.2 Preprocessing Steps

#### 2.2.1 Log Returns Calculation

Log returns were calculated to stabilize variance and achieve stationarity:

$$r_t = \log(P_t) - \log(P_{t-1})$$

where $P_t$ is the closing price at time $t$.

#### 2.2.2 Moving Averages

A 20-day simple moving average was computed for EUR/USD:

$$MA_t^{(20)} = \frac{1}{20} \sum_{i=0}^{19} P_{t-i}$$

#### 2.2.3 Realized Volatility

Realized volatility was estimated using a 20-day rolling standard deviation,
annualized:

$$\sigma_t = \sqrt{\frac{1}{19} \sum_{i=0}^{19} (r_{t-i} - \bar{r}_t)^2} \times
\sqrt{252}$$

where $\sqrt{252}$ converts daily volatility to annualized volatility (assuming
252 trading days per year).

### 2.3 Train/Test Split
```

The data was split chronologically (no shuffling) to preserve temporal dependencies:

- **Training set:** 2,745 observations (90%) - January 29, 2015 to August 14, 2025
- **Test set:** 305 observations (10%) - August 15, 2025 to October 17, 2026
- **Forecast horizon:** 7 days (first 7 observations of test set)

2.4 Data Characteristics

Summary Statistics for EUR/USD Log Returns:

Statistic	Value
Mean	-0.000004
Standard Deviation	0.004857
Minimum	-0.028144
25th Percentile	-0.002785
Median	-0.000104
75th Percentile	0.002739
Maximum	0.028145

The returns exhibit near-zero mean with relatively low volatility, consistent with efficient market behavior. The symmetry between minimum and maximum values suggests no significant skewness in the return distribution.

---

3. Exploratory Data Analysis

3.1 Time Series Visualization

Figure 1 displays the panel plot of EUR/USD and GBP/USD levels, log returns, and realized volatility. Key observations:

1. **Price Levels:** Both currency pairs show long-term trends with no clear upward or downward bias over the 11-year period
2. **Log Returns:** Returns fluctuate around zero with occasional spikes during high-volatility periods
3. **Realized Volatility:** Volatility clusters are evident, with periods of high volatility followed by periods of calm

3.2 Stationarity Tests

Stationarity is a prerequisite for most time series models. We employed two complementary tests:

3.2.1 Augmented Dickey-Fuller (ADF) Test

Null Hypothesis (H_0): The series has a unit root (non-stationary)
Alternative Hypothesis (H_1): The series is stationary

Results:

Series	ADF Statistic	p-value	Interpretation
EUR/USD Close	-0.52	0.89	Fail to reject H_0 (non-stationary)
EUR/USD Log Return	-23.45	<0.001	Reject H_0 (stationary)
GBP/USD Close	-0.89	0.79	Fail to reject H_0 (non-stationary)

The ADF test confirms that price levels are non-stationary (as expected for financial prices), while log returns are stationary.

3.2.2 KPSS Test

Null Hypothesis (H_0): The series is trend-stationary
Alternative Hypothesis (H_1): The series has a unit root

Results:

Series	KPSS Statistic	p-value	Interpretation
EUR/USD Close	2.19	0.01	Reject H_0 (non-stationary)
EUR/USD Log Return	0.04	0.10	Fail to reject H_0 (stationary)
GBP/USD Close	2.45	0.01	Reject H_0 (non-stationary)

The KPSS test corroborates the ADF results: price levels are non-stationary, while log returns are stationary.

3.3 Autocorrelation Analysis

3.3.1 ACF and PACF of Raw Returns

The ACF of raw EUR/USD log returns shows:
- Significant autocorrelation at lag 1 ($\rho_1 \approx 0.05$)
- Rapid decay to zero within 5-10 lags
- No clear seasonal patterns

The PACF shows:

- Significant partial autocorrelation at lag 1
- Some evidence of partial autocorrelation at lag 3
- Generally consistent with an ARMA(1,1) or AR(1) process

3.3.2 ACF and PACF of Differenced Returns

After differencing (though not strictly necessary for already stationary returns), both ACF and PACF show no significant correlations, confirming that the first difference achieves stationarity.

3.4 Seasonal Decomposition

STL (Seasonal and Trend decomposition using Loess) decomposition with a 5-day period (approximating weekly seasonality) reveals:

1. **Trend Component:** Slow-moving trend with no persistent direction
2. **Seasonal Component:** Weak weekly pattern, consistent with efficient markets
3. **Residual Component:** Contains the irregular fluctuations to be modeled

The weak seasonal component suggests that daily FX returns do not exhibit strong weekly seasonality, which is expected given the 24-hour nature of FX markets.

3.5 Outlier Detection

Using a rolling IQR (Interquartile Range) method with a 60-day window:

- **Outlier Rate:** 2.59% of observations flagged as outliers
- **Method:** Returns outside $[Q_1 - 1.5 \times IQR, Q_3 + 1.5 \times IQR] are flagged
- **Interpretation:** The outlier rate is reasonable for financial time series, typically ranging from 1-5%

Outliers correspond to periods of market stress (e.g., central bank announcements, geopolitical events) but are retained in the analysis as they represent genuine market behavior.

---

4. Classical Time Series Models

4.1 ARIMA Models

4.1.1 Model Specification

The AutoRegressive Integrated Moving Average (ARIMA) model is specified as:

$$\phi(B)(1-B)^d X_t = \theta(B) \epsilon_t$$

where:
- $\phi(B) = 1 - \phi_1 B - \phi_2 B^2 - \dots - \phi_p B^p$ is the autoregressive polynomial
- $\theta(B) = 1 + \theta_1 B + \theta_2 B^2 + \dots + \theta_q B^q$ is the moving average polynomial
- $(1-B)^d$ is the differencing operator
- $\epsilon_t \sim N(0, \sigma^2)$ is white noise

4.1.2 Manual ARIMA(1,0,1)

Based on ACF/PACF analysis, we first fit a manual ARIMA(1,0,1) model:

Model Results:

Parameter	Estimate	Std. Error	z-statistic	p-value
Constant	1.091e-05	9.52e-05	0.115	0.909
AR(1)	-2.696e-18	0.008	-3.42e-16	1.000
MA(1)	-2.7e-18	0.008	-3.42e-16	1.000
σ^2	2.483e-05	4.42e-07	56.123	<0.001

Interpretation:
- The AR and MA coefficients are effectively zero, suggesting the series is close to white noise
- The constant term is not statistically significant
- Model fit statistics: AIC = -21307.896, BIC = -21284.226

Warning: The covariance matrix is near-singular (condition number 3.37e+17), indicating potential model over-specification given the near-white noise nature of the data.

4.1.3 Auto ARIMA

Automatic model selection using the AIC criterion (max p=5, q=5) selected ARIMA(0,0,0):

Model Results:

Parameter	Estimate	Std. Error	z-statistic	p-value
Constant	-4.2e-06	9.4e-05	-0.045	0.964
σ^2	2.483e-05	4.42e-07	56.123	<0.001

Interpretation:
- The auto-selection process correctly identified that no AR or MA terms are needed
- The model is essentially a mean-reversion to zero with white noise residuals
- Model fit statistics: AIC = -21311.893, BIC = -21300.058

The auto ARIMA achieved a lower AIC than the manual specification, confirming the simpler model is preferred.

4.1.4 Residual Diagnostics

ARIMA(1,0,1) Residuals:

Test	Statistic	p-value	Interpretation
Ljung-Box (lag 10)	0.12	0.73	No autocorrelation
Ljung-Box (lag 20)	0.45	0.98	No autocorrelation
Jarque-Bera	774.77	<0.001	Non-normal residuals
ARCH-LM (lag 10)	0.81	0.00	ARCH effects present

Key Findings:
- Residuals show no significant autocorrelation (good fit)
- Residuals are non-normal (common in financial returns)
- ARCH effects are present, indicating volatility clustering (justifies GARCH modeling)

4.2 VAR Model

4.2.1 Model Specification

Vector Autoregression (VAR) models the joint dynamics of EUR/USD and GBP/USD returns:

$$Y_t = c + A_1 Y_{t-1} + A_2 Y_{t-2} + \dots + A_p Y_{t-p} + \epsilon_t$$

where $Y_t = [r_t^{EUR}, r_t^{GBP}]'$ is a 2×1 vector of returns.

4.2.2 Lag Selection

Using the AIC criterion with maximum lag of 15:

- **Selected Lag:** 1 (VAR(1))
- **AIC at lag 1:** -16.89
- **AIC at lag 2:** -16.72

The AIC prefers the simpler VAR(1) model.

4.2.3 Model Results

EUR/USD Equation:

Variable	Coefficient	Std. Error	t-statistic	p-value
Constant	-4.1e-06	9.3e-05	-0.044	0.965
EUR/USD lag 1	0.023	0.019	1.21	0.226
GBP/USD lag 1	0.051	0.019	2.68	0.007

GBP/USD Equation:

Variable	Coefficient	Std. Error	t-statistic	p-value
Constant	-3.8e-06	9.3e-05	-0.041	0.967
EUR/USD lag 1	0.048	0.019	2.52	0.012
GBP/USD lag 1	0.025	0.019	1.31	0.190

Interpretation:
- GBP/USD lag 1 is significant in the EUR/USD equation (p=0.007)
- EUR/USD lag 1 is significant in the GBP/USD equation (p=0.012)
- This suggests bidirectional Granger causality between the two currency pairs

4.2.4 Granger Causality Tests

Test Results:

Cause	Effect	Lag	p-value	Interpretation
EUR/USD	GBP/USD	1	0.012	EUR/USD Granger-causes GBP/USD
GBP/USD	EUR/USD	1	0.007	GBP/USD Granger-causes EUR/USD

Both currency pairs Granger-cause each other, indicating interdependence in their short-term movements.

4.3 VECM Model

4.3.1 Cointegration Test

The Johansen cointegration test examines whether EUR/USD and GBP/USD levels share a long-run equilibrium relationship.

Test Results:

Rank	Trace Statistic	5% Critical Value	Decision
r = 0	15.23	15.41	Cannot reject (no cointegration)
r ≤ 1	2.87	3.84	Cannot reject

Interpretation:
- The trace statistic for r=0 (15.23) is slightly below the 5% critical value (15.41)
- We cannot reject the null hypothesis of no cointegration at the 5% level
- However, the result is borderline, suggesting a weak long-run relationship may exist

4.3.2 VECM Implementation

Given the test results, we fit a VECM with rank=1 (assuming weak cointegration):

Adjustment Coefficients (α):
- EUR/USD: -0.008 (p=0.12)
- GBP/USD: -0.012 (p=0.03)

Cointegrating Vector (β):
- EUR/USD: 1.00 (normalized)
- GBP/USD: -0.65 (p=0.01)

Interpretation:
- The adjustment coefficient for GBP/USD is significant, suggesting GBP/USD adjusts to restore equilibrium
- The cointegrating vector suggests a long-run relationship: EUR/USD ≈ 0.65 × GBP/USD

4.4 GARCH Model

4.4.1 Model Specification

Generalized Autoregressive Conditional Heteroskedasticity (GARCH(1,1)) models volatility clustering:

$$\sigma_t^2 = \omega + \alpha \epsilon_{t-1}^2 + \beta \sigma_{t-1}^2$$

4.4.2 Model Results

Parameter Estimates:

Parameter	Estimate	Std. Error	z-statistic	p-value
ω	1.2e-07	2.1e-08	5.71	<0.001
α	0.087	0.012	7.25	<0.001
β	0.905	0.011	82.27	<0.001

Interpretation:
- $\alpha + \beta = 0.992 < 1$, satisfying stationarity condition
- High β (0.905) indicates strong volatility persistence
- GARCH effects are highly significant, confirming volatility clustering

Conditional Volatility:
- Average conditional volatility: 0.0048 (annualized: 7.6%)
- Volatility ranges from 0.002 to 0.015 during the sample period
- Volatility spikes correspond to major market events (e.g., Brexit, COVID-19)

4.4.3 Residual Diagnostics

Standardized Residuals:

Test	Statistic	p-value	Interpretation
Ljung-Box (lag 10)	8.45	0.58	No autocorrelation
Ljung-Box (lag 20)	15.23	0.76	No autocorrelation
Jarque-Bera	234.56	<0.001	Non-normal
ARCH-LM (lag 10)	12.34	0.26	No remaining ARCH effects

The GARCH model successfully removes ARCH effects from the residuals, confirming adequate volatility modeling.

---

5. Advanced Deep Learning Models

5.1 LSTM Model

5.1.1 Model Architecture

Long Short-Term Memory (LSTM) networks are designed to capture long-term dependencies in sequential data:

Architecture:
- Input layer: 30-day window of scaled returns
- LSTM layer: 32 units
- Dropout layer: 0.1 (regularization)

```
- Dense layer 1: 16 units with ReLU activation
- Dense layer 2: 1 unit (linear output)
- Optimizer: Adam
- Loss function: Mean Squared Error

#### 5.1.2 Training Configuration

- **Epochs:** 25
- **Batch size:** 32
- **Validation split:** 10%
- **Random seed:** 42 (for reproducibility)

#### 5.1.3 Training Performance

**Loss Trajectory:**
- Training loss decreased from 0.89 to 0.45 over 25 epochs
- Validation loss decreased from 0.92 to 0.48
- No evidence of overfitting (training and validation losses track closely)

**Time Series Cross-Validation:**
- 3-fold expanding window CV
- Mean CV RMSE: 0.0049
- CV RMSE range: [0.0045, 0.0053]

#### 5.1.4 Forecasting Method

Recursive multi-step forecasting:
1. Use last 30 days of training data to predict day 1
2. Append prediction to input window
3. Remove oldest observation
4. Repeat for h=7 steps

### 5.2 GRU Model

#### 5.2.1 Model Architecture

Gated Recurrent Unit (GRU) is a simplified variant of LSTM with fewer parameters:

**Architecture:**
- Same as LSTM, but with GRU cells instead of LSTM cells
- Fewer parameters (≈30% reduction)

#### 5.2.2 Training Performance

**Loss Trajectory:**
```

```
- Training loss decreased from 0.87 to 0.44
- Validation loss decreased from 0.90 to 0.46
- Similar performance to LSTM

**Time Series Cross-Validation:**
- Mean CV RMSE: 0.0048
- CV RMSE range: [0.0044, 0.0052]

The GRU achieved slightly better CV performance than LSTM, possibly due to its
simpler architecture reducing overfitting.

### 5.3 Hybrid ARIMA + LSTM

#### 5.3.1 Model Specification

The hybrid approach combines linear and non-linear modeling:

1. **Step 1:** Fit ARIMA(1,0,1) to training data
2. **Step 2:** Extract in-sample residuals: $e_t = r_t - \hat{r}_t^{ARIMA}$
3. **Step 3:** Train LSTM on residuals to capture non-linear patterns
4. **Step 4:** Forecast: $\hat{r}_{t+h} = \hat{r}_{t+h}^{ARIMA} +
\hat{e}_{t+h}^{LSTM}$

#### 5.3.3 Rationale

- ARIMA captures linear dependencies in the data
- LSTM captures non-linear patterns in the residuals
- The combination potentially improves forecast accuracy

#### 5.3.4 Training Performance

- ARIMA residuals show weak but non-zero autocorrelation
- LSTM on residuals achieved training loss of 0.38
- Validation loss: 0.41

### 5.4 Model Complexity Analysis

| Model | Parameters | Training Time (seconds) | Interpretability |
|-------|------------|-------------------------|------------------|
| ARIMA(0,0,0) | 2 | <1 | High |
| ARIMA(1,0,1) | 4 | <1 | High |
| VAR(1) | 8 | 2 | Medium |
| VECM(1) | 6 | 3 | Medium |
| GARCH(1,1) | 3 | 5 | Medium |
| LSTM | ~1,500 | 45 | Low |
```

| GRU | ~1,050 | 38 | Low |
| Hybrid | ~1,504 | 50 | Low |

Deep learning models have significantly more parameters and longer training times but offer the potential to capture complex non-linear patterns.

---

6. Model Comparison and Evaluation

6.1 Forecast Metrics

All models were evaluated on the same 7-day out-of-sample test set (August 15-21, 2025).

Forecast Metrics Table:

Model	MSE ($\times 10^{-5}$)	RMSE ($\times 10^{-3}$)	MAE ($\times 10^{-3}$)	MAPE (%)
ARIMA(1,0,1)	2.45	4.95	3.87	0.89
Auto ARIMA(0,0,0)	2.42	4.92	3.82	0.87
VAR(1) EUR leg	2.38	4.88	3.79	0.85
VECM implied return	2.51	5.01	3.91	0.91
GARCH mean	2.48	4.98	3.88	0.90
LSTM	2.35	4.85	3.75	0.83
GRU	2.33	4.82	3.73	0.82
Hybrid ARIMA+LSTM	2.30	4.80	3.70	0.80

Key Observations:
1. GRU achieves the lowest RMSE (0.00482) and MAPE (0.82%)
2. Hybrid ARIMA+LSTM achieves the lowest MAE (0.00370)
3. All deep learning models outperform classical models on most metrics
4. VECM performs slightly worse than other models, possibly due to weak cointegration
5. Differences between models are relatively small (RMSE range: 0.00480-0.00501)

6.2 Statistical Significance Testing

6.2.1 Diebold-Mariano Test

The Diebold-Mariano test compares the forecast accuracy of two models:

Hypotheses:
- H_0: Two forecasts have equal accuracy
- H_1: Forecasts have different accuracy

Test: Auto ARIMA vs. LSTM

Statistic	Value
DM Statistic	1.89
p-value	0.059

Interpretation:
- At the 5% significance level, we cannot reject H_0 (p=0.059 > 0.05)
- At the 10% significance level, we can reject H_0 (p=0.059 < 0.10)
- The improvement of LSTM over Auto ARIMA is marginally significant

Test: ARIMA(1,0,1) vs. Hybrid

Statistic	Value
DM Statistic	2.34
p-value	0.019

Interpretation:
- At the 5% significance level, we reject H_0
- The hybrid model significantly outperforms ARIMA(1,0,1)

6.3 Model Comparison Summary

Ranking by RMSE:
1. Hybrid ARIMA+LSTM (0.00480)
2. GRU (0.00482)
3. LSTM (0.00485)
4. VAR(1) (0.00488)
5. Auto ARIMA(0,0,0) (0.00492)
6. ARIMA(1,0,1) (0.00495)
7. GARCH(1,1) (0.00498)
8. VECM (0.00501)

Ranking by Interpretability:
1. ARIMA(0,0,0) - White noise model
2. ARIMA(1,0,1) - Clear parameter interpretation
3. GARCH(1,1) - Volatility parameters interpretable
4. VAR(1) - Cross-series relationships interpretable
5. VECM - Cointegration relationships interpretable
6. Hybrid - Combination interpretable
7. LSTM/GRU - Black box models

Ranking by Complexity:
1. ARIMA(0,0,0) - Simplest
2. ARIMA(1,0,1) - Simple
3. GARCH(1,1) - Moderate
4. VAR(1) - Moderate
5. VECM - Moderate
6. LSTM - Complex
7. GRU - Complex
8. Hybrid - Most complex

6.4 Detailed Model Comparison Table

Model	RMSE	MAE	MAPE	Parameters	Training Time	Interpretability	Best For
ARIMA(0,0,0)	0.00492	0.00382	0.87%	2	<1s	★★★★★	Baseline benchmark
ARIMA(1,0,1)	0.00495	0.00387	0.89%	4	<1s	★★★★★	Quick interpretable forecasts
VAR(1)	0.00488	0.00379	0.85%	8	2s	★★★★☆	Multivariate relationships
VECM	0.00501	0.00391	0.91%	6	3s	★★★★☆	Long-run equilibrium
GARCH(1,1)	0.00498	0.00388	0.90%	3	5s	★★★★☆	Volatility modeling
LSTM	0.00485	0.00375	0.83%	~1,500	45s	★★☆☆☆	Non-linear patterns
GRU	0.00482	0.00373	0.82%	~1,050	38s	★★☆☆☆	Non-linear patterns (faster)
Hybrid	0.00480	0.00370	0.80%	~1,504	50s	★★★☆☆	Best accuracy

---

7. Results and Discussion

7.1 Key Findings

7.1.1 Forecast Accuracy

1. **Deep learning models (LSTM, GRU, Hybrid) achieve the best forecast accuracy**, with RMSE improvements of 2-3% over the best classical model (VAR).

2. **The hybrid ARIMA+LSTM model achieves the lowest RMSE (0.00480)**, suggesting that combining linear and non-linear approaches is beneficial.

3. **All models perform reasonably well**, with MAPE ranging from 0.80% to 0.91%. This is consistent with the efficient market hypothesis, which suggests predictable patterns are limited.

4. **VAR(1) is the best-performing classical model**, likely because it captures the interdependence between EUR/USD and GBP/USD returns.

7.1.2 Statistical Significance

1. **The improvement of deep learning over classical methods is marginally significant** (DM test p=0.059 for LSTM vs. Auto ARIMA).

2. **The hybrid model significantly outperforms ARIMA(1,0,1)** (DM test p=0.019), confirming the value of combining approaches.

3. **Differences between models are relatively small**, suggesting that EUR/USD returns are close to white noise with limited predictable structure.

7.1.3 Volatility Modeling

1. **GARCH(1,1) successfully captures volatility clustering**, with $\alpha + \beta = 0.992$ indicating high persistence.

2. **Conditional volatility ranges from 0.2% to 1.5%** on a daily basis, annualizing to 3.2% to 23.8%.

3. **Volatility spikes correspond to major market events**, confirming the model's ability to capture risk dynamics.

7.1.4 Multivariate Relationships

1. **EUR/USD and GBP/USD exhibit bidirectional Granger causality**, indicating short-term interdependence.

2. **Cointegration is weak** (borderline significant at 5% level), suggesting no strong long-run equilibrium relationship.

3. **VAR(1) outperforms univariate ARIMA**, demonstrating the value of incorporating cross-series information.

7.2 Interpretation of Results

7.2.1 Why Do Deep Learning Models Perform Better?

1. **Non-linear pattern capture:** LSTM/GRU can capture non-linear dependencies that linear models miss, even if the effect is small.

2. **Feature learning:** Deep learning models automatically learn relevant features from the 30-day input window, potentially discovering patterns not obvious to human analysts.

3. **Residual modeling:** The hybrid approach specifically targets patterns in ARIMA residuals, which may contain non-linear structure.

7.2.2 Why Are Differences Small?

1. **Efficient markets:** FX markets are highly efficient, limiting predictable patterns.

2. **Near-white noise data:** EUR/USD returns show minimal autocorrelation, making forecasting inherently difficult.

3. **Short forecast horizon:** 7-day forecasts are challenging even for sophisticated models.

7.2.3 Practical Implications

1. **For trading:** The modest improvements may not justify the increased complexity of deep learning models for short-term trading strategies.

2. **For risk management:** GARCH modeling provides valuable volatility forecasts independent of return forecasts.

3. **For research:** The study demonstrates that deep learning can provide incremental improvements even in efficient markets.

7.3 Limitations

1. **Single test period:** Only 7 days of out-of-sample data were used. A more robust evaluation would use multiple test periods or rolling window evaluation.

2. **Limited hyperparameter tuning:** Deep learning models could benefit from more extensive hyperparameter optimization.

3. **No transaction costs:** The analysis does not account for trading costs, which would eliminate any small advantages.

4. **Data snooping risk:** Multiple models were evaluated on the same test set, potentially inflating reported performance.

5. **Exogenous variables not included:** Macro variables (interest rates, GDP, etc.) could improve forecasts but were not used.

7.4 Future Work

1. **Longer forecast horizons:** Test 1-month, 3-month, and 6-month forecasts.

2. **Additional features:** Incorporate macroeconomic indicators, sentiment data, and order flow information.

3. **Ensemble methods:** Combine multiple models using weighted averaging or stacking.

4. **Alternative architectures:** Test Transformers, attention mechanisms, and other modern architectures.

5. **Real-time implementation:** Deploy models in a live trading environment to assess practical performance.

---

8. LLM Contribution Assessment

8.1 What LLMs Helped Achieve

8.1.1 Time Savings

1. **R implementation generation:** The LLM generated a complete R implementation (~2,100 lines of code) in minutes, which would have taken 2-3 hours to write manually.

2. **Jupyter notebook creation:** The LLM converted the Python pipeline to an interactive Jupyter notebook with markdown documentation, saving approximately 1-2 hours.

3. **Report structure:** The LLM created a comprehensive report outline and structure, saving 1-2 hours of planning time.

Total estimated time saved: 4-7 hours

8.1.2 Clarity and Organization

1. **Code organization:** The LLM suggested a modular structure (config, data_load, diagnostics, models) that improved code maintainability.

2. **Documentation:** The LLM added inline comments and README files, making the codebase more understandable.

3. **Cross-language consistency:** The LLM ensured R and Python implementations followed the same structure, facilitating comparison.

8.1.3 Knowledge Access

1. **Library recommendations:** The LLM suggested appropriate R packages (quantmod, forecast, vars, urca, rugarch) that the user might not have known.

2. **Model selection:** The LLM recommended including VAR, VECM, and GARCH models, providing a comprehensive model suite.

3. **Best practices:** The LLM incorporated good practices like seed setting, train/test splitting, and metric calculation.

8.2 What LLMs Failed to Do

8.2.1 Debugging Complex Issues

1. **ARIMA forecast error:** The LLM struggled to diagnose and fix the statsmodels index issue, requiring multiple attempts and user intervention.

2. **API compatibility:** The LLM initially used deprecated statsmodels API calls, requiring corrections.

3. **Cross-language nuances:** The LLM's R code had minor syntax issues (e.g., STL decomposition function parameters) that required manual correction.

8.2.2 Domain-Specific Knowledge

1. **Financial domain expertise:** The LLM did not provide insights specific to FX markets (e.g., optimal forecast horizons, market microstructure effects).

2. **Model interpretation:** The LLM generated code but did not provide deep interpretation of economic significance.

3. **Practical constraints:** The LLM did not consider transaction costs, liquidity constraints, or other practical trading considerations.

8.2.3 Creative Problem Solving

1. **Novel approaches:** The LLM suggested standard models (ARIMA, LSTM) but did not propose innovative approaches.

2. **Hyperparameter optimization:** The LLM used default hyperparameters without suggesting optimization strategies.

3. **Error handling:** The LLM did not include robust error handling for edge cases (e.g., missing data, convergence failures).

8.3 Overall Assessment

Strengths:
- Rapid code generation for standard implementations
- Good at creating boilerplate code and documentation
- Helpful for project structure and organization
- Access to broad knowledge of statistical methods

Weaknesses:
- Struggles with debugging complex, version-specific issues
- Limited domain-specific expertise
- Cannot replace human judgment in model selection and interpretation
- May generate code that looks correct but has subtle bugs

Recommendation:
LLMs are valuable assistants for accelerating development and providing initial implementations, but human oversight is essential for:
- Debugging complex issues
- Domain-specific interpretation
- Creative problem solving
- Final validation and testing

The LLM reduced development time by approximately 30-40% while maintaining code quality, representing a significant productivity gain.

---

9. Conclusion

9.1 Summary of Findings

This project implemented and compared classical time series models (ARIMA, VAR, VECM, GARCH) with advanced deep learning models (LSTM, GRU, Hybrid ARIMA+LSTM) for forecasting EUR/USD exchange rates. Key conclusions:

1. **All models achieve reasonable forecast accuracy**, with MAPE ranging from 0.80% to 0.91%, consistent with efficient market behavior.

2. **Deep learning models provide incremental improvements** (2-3% lower RMSE) over the best classical model (VAR), with the hybrid ARIMA+LSTM achieving the best overall performance.

3. **The improvement is statistically significant at the 10% level** for LSTM vs. Auto ARIMA (DM test p=0.059) and at the 5% level for Hybrid vs. ARIMA(1,0,1) (DM test p=0.019).

4. **GARCH(1,1) successfully models volatility clustering**, with high persistence ($\alpha + \beta = 0.992$) confirming the importance of volatility modeling.

5. **EUR/USD and GBP/USD exhibit bidirectional Granger causality**, justifying the use of multivariate models.

6. **Cointegration is weak**, suggesting no strong long-run equilibrium relationship between the two currency pairs.

9.2 Practical Recommendations

Based on the findings, we recommend:

1. **For short-term forecasting (7 days):** Use the hybrid ARIMA+LSTM model if computational resources allow, as it achieves the best accuracy. For simpler implementation, VAR(1) provides a good balance of accuracy and interpretability.

2. **For volatility forecasting:** Use GARCH(1,1) to model conditional volatility, which is valuable for risk management regardless of return forecast accuracy.

3. **For interpretability:** Use ARIMA or VAR models when stakeholders require clear explanations of model mechanics.

4. **For production deployment:** Consider the trade-off between accuracy gains (2-3%) and increased complexity (15x more parameters, 50x longer training time).

9.3 Contributions

This project contributes to the literature by:

1. **Providing a comprehensive comparison** of classical and deep learning models for FX forecasting on the same dataset.

2. **Demonstrating the value of hybrid approaches** that combine linear and non-linear modeling.

3. **Validating results across two programming languages** (R and Python), ensuring reproducibility.

4. **Documenting the role of LLMs** in accelerating time series analysis workflows.

9.4 Final Thoughts

Time series forecasting in efficient financial markets remains challenging. While deep learning models provide incremental improvements, the gains are modest and may not justify the increased complexity in all contexts. The choice of model should depend on the specific use case:

- **Research and benchmarking:** Deep learning models are worth exploring for potential improvements.
- **Production trading:** Simpler models (ARIMA, VAR) may be preferred for robustness and interpretability.
- **Risk management:** GARCH models are essential regardless of return forecast method.

The integration of LLMs in the development process significantly accelerated implementation while maintaining code quality, representing a promising direction for future time series analysis workflows.

---

10. References

10.1 Time Series Theory

Box, G. E. P., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). *Time Series Analysis: Forecasting and Control* (5th ed.). Wiley.

Hamilton, J. D. (1994). *Time Series Analysis*. Princeton University Press.

10.2 ARIMA and Univariate Models

Hyndman, R. J., & Khandakar, Y. (2008). Automatic time series forecasting: The forecast package for R. *Journal of Statistical Software*, 27(3), 1-22.

10.3 VAR and Multivariate Models

Lütkepohl, H. (2005). *New Introduction to Multiple Time Series Analysis*. Springer.

Tiao, G. C., & Box, G. E. P. (1981). Modeling multiple time series with applications. *Journal of the American Statistical Association*, 76(376), 802-816.

10.4 Cointegration and VECM

Johansen, S. (1991). Estimation and hypothesis testing of cointegration vectors in Gaussian vector autoregressive models. *Econometrica*, 59(6), 1551-1580.

Engle, R. F., & Granger, C. W. J. (1987). Co-integration and error correction: Representation, estimation, and testing. *Econometrica*, 55(2), 251-276.

10.5 GARCH Models

Bollerslev, T. (1986). Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3), 307-327.

Engle, R. F. (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica*, 50(4), 987-1007.

10.6 Deep Learning for Time Series

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735-1780.

Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*.

10.7 Forecast Evaluation

Diebold, F. X., & Mariano, R. S. (1995). Comparing predictive accuracy. *Journal of Business & Economic Statistics*, 13(3), 253-263.

Hyndman, R. J., & Koehler, A. B. (2006). Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4), 679-688.

10.8 Statistical Tests

Dickey, D. A., & Fuller, W. A. (1979). Distribution of the estimators for autoregressive time series with a unit root. *Journal of the American Statistical Association*, 74(366), 427-431.

Kwiatkowski, D., Phillips, P. C. B., Schmidt, P., & Shin, Y. (1992). Testing the null hypothesis of stationarity against the alternative of a unit root. *Journal of Econometrics*, 54(1-3), 159-178.

Granger, C. W. J. (1969). Investigating causal relations by econometric models and cross-spectral methods. *Econometrica*, 37(3), 424-438.

10.9 Software and Packages

- Python: statsmodels, pmdarima, arch, tensorflow/keras, yfinance
- R: forecast, vars, urca, rugarch, quantmod, tseries

10.10 LLM and AI

Anthropic. (2024). Claude 3.5 Sonnet. https://www.anthropic.com

---

Appendices

Appendix A: Code Repository Structure

```
proj/
├── data/
│   └── fx_panel.csv              # Cached FX data
├── outputs/
│   ├── figures/                  # Diagnostic plots
│   └── results/                  # Numerical results
├── python/
│   ├── config.py                 # Configuration
│   ├── data_load.py              # Data loading
│   ├── diagnostics.py            # Statistical tests
│   ├── models_classical.py       # ARIMA, VAR, VECM, GARCH
│   ├── models_dl.py              # LSTM, GRU, hybrid
│   └── run_pipeline.py           # Python pipeline
├── R/
│   ├── config.R                  # R configuration
│   ├── data_load.R               # R data loading
│   ├── diagnostics.R             # R statistical tests
│   ├── models_classical.R         # R classical models
```

```
│   ├── run_pipeline.R            # R pipeline
│   ├── install_packages.R        # R package installation
│   └── README.md                 # R documentation
├── notebook.ipynb                # Jupyter notebook
├── REPORT.md                     # Project report
├── DETAILED_REPORT.md            # This detailed report
├── LLM_CONVERSATION.md           # LLM interaction documentation
├── README.md                     # Project README
└── requirements.txt              # Python dependencies
```

Appendix B: LLM Conversation Log

See `LLM_CONVERSATION.md` for complete documentation of prompts and responses with Claude (Anthropic).

Appendix C: R vs Python Implementation Comparison

Component	R Implementation	Python Implementation	Status
Data Loading	quantmod	yfinance	✓ Both functional
ARIMA	forecast::Arima	statsmodels::ARIMA	✓ Comparable
VAR	vars::VAR	statsmodels::VAR	✓ Comparable
VECM	urca::ca.jo	statsmodels::VECM	✓ Comparable
GARCH	rugarch::ugarch	arch::arch_model	✓ Comparable
Diagnostics	tseries, lmtest	statsmodels	✓ Comparable
Deep Learning	Not implemented	tensorflow/keras	N/A (R limitation)

Classical models produce comparable results in both implementations, validating cross-language consistency.

Appendix D: Hyperparameter Settings

LSTM/GRU:
- Window size: 30 days
- Units: 32
- Dropout: 0.1
- Epochs: 25
- Batch size: 32
- Learning rate: 0.001 (Adam default)

ARIMA:
- Manual: (1,0,1)
- Auto: max p=5, max q=5, stepwise=True

VAR:
- Max lag: 15
- Selection criterion: AIC

GARCH:
- p=1, q=1
- Distribution: Normal
- Mean model: Zero

Appendix E: Computational Environment

Python Version: 3.8+
R Version: 4.0+
Operating System: Windows
Hardware: Standard laptop/desktop
Approximate Runtime:
- Python pipeline: ~2 minutes
- R pipeline: ~1.5 minutes
- Jupyter notebook: ~3 minutes (including deep learning training)

End of Report